Boosting Compiler Fault Localization: Getting the Best of Both Worlds by Fusing Dynamic and Historical Data

Qingyang Li, Yibiao Yang, Maolin Sun, Jiangchang Wu, Qingkai Shi, Yuming Zhou, and Baowen Xu

***Abstract*—Compilers are prone to bugs that can have severe consequences for downstream applications. Accurately identifying and localizing compiler faults poses unique challenges due to the inherent complexity and large scale of modern compiler infrastructures. Existing studies have proposed various techniques to construct passing and failing executions by generating witness test programs from bug-inducing test cases or by producing adversarial compilation configurations for the same test program. These executions are then leveraged to apply spectrum-based fault localization (SBFL) techniques for isolating compiler faults, yielding promising results. Recently, Yang et al. revisited SBFL-based techniques and showed that a simple yet widely adopted debugging practice—treating files modified in bug-inducing commits (BICs) as potential fault candidates—can surprisingly outperform SBFL-based techniques on the most critical localization metrics. Moreover, they further demonstrated that BIC-based and SBFL-based techniques are highly complementary, as they tend to localize different subsets of compiler faults. Consequently, effectively integrating these two sources of information to improve compiler fault localization remains an open and largely unexplored challenge. To address this problem, we propose DUALTRACK, a hybrid approach that integrates dynamic execution information from SBFL with historical information derived from BICs. DUALTRACK employs a two-layer framework that first prioritizes files modified in bug-inducing commits and then refines their rankings using suspiciousness scores computed by SBFL formulas. An evaluation on 120 real-world compiler bugs from GCC and LLVM shows that DUALTRACK successfully identifies 52% of faulty files at the Top-1 rank, demonstrating a substantial improvement over existing state-of-the-art compiler fault localization techniques.**

***Index Terms*—Compilers, fault localization, bug-inducing commit.**

Received 25 July 2025; revised 31 December 2025; accepted 16 February 2026. Date of publication 19 February 2026; date of current version 20 April 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 624B2067, Grant 62072194, Grant 62572226, and Grant 62472215, in part by Jiangsu Natural Science Foundation under Grant BK20231402, in part by the Collaborative Innovation Center of Novel Software Technology and Industrialization. Recommended for acceptance by G. Denaro. *(Corresponding author: Yibiao Yang.)*

The authors are with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: liqingyang@smail.nju.edu.cn; yangyibiao@nju.edu.cn; merlin@smail.nju.edu.cn; jiangchangwu@smail.nju.edu.cn; qingkaishi@nju.edu.cn; zhouyuming@nju.edu.cn; bwxu@nju.edu.cn).

Digital Object Identifier 10.1109/TSE.2026.3666208

I. INTRODUCTION

COMPILERS are fundamental components in software development, translating high-level code into executable machine code [1]. Faults in compilers can result in incorrect binaries, leading developers to waste substantial debugging effort on inspecting their application code rather than identifying the compiler defects. Consequently, accurately detecting and localizing compiler faults is essential for ensuring software reliability [2]. Modern compilers comprise highly complex components, including optimization passes, code transformations, and target-specific code generators, which together pose significant challenges for fault localization [3].

To address these challenges, automated fault localization techniques, most notably Spectrum-Based Fault Localization (SBFL), have been extensively studied and applied in software debugging [4]. Several SBFL techniques have been specifically designed for isolating compiler faults [3], [5], [6], [7], [8], [9]. SBFL is based on the intuition that execution profiles of passing and failing test cases provide valuable clues about fault locations. By constructing a coverage spectrum from these executions, SBFL ranks code elements (e.g., files, functions, or statements) according to their suspiciousness, which reflects the likelihood that a given element is responsible for the observed failure. Commonly used SBFL formulas, such as Ochiai [10] and Dstar [11], are employed to compute these scores. However, SBFL often suffers from limited precision when coverage information is sparse or when faults arise from complex interdependencies among components [12], [13]. For example, even advanced SBFL techniques such as ODFL localize only 36 out of 120 real-world GCC and LLVM bugs at the top-ranked file (Top-1), leaving more than 50% of faults outside the top five ranked files (Top-5).

In practice, as emphasized by Yang et al. [14], compiler developers often rely on version control systems to track code changes and identify the specific commit responsible for introducing a bug. Commit-based techniques, such as git bisect, are widely adopted in both the LLVM [15] and GCC [16] communities to isolate bug-inducing commits (BICs). A bug-inducing commit is the commit that first introduces the code change responsible for a subsequently observed bug. In the context of compilers, the compiler behaves correctly for a given

0098-5589 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

test program prior to the commit, but exhibits incorrect behavior after the commit is applied. By identifying the BIC, developers can substantially narrow the fault localization search space to the files or functions modified in that commit. Despite their practical effectiveness, commit-based approaches suffer from inherent limitations. In particular, some bugs may already exist in the codebase but remain dormant until later changes trigger their manifestation. In such cases, the identified commit merely exposes a previous fault rather than introducing it. Consequently, the files modified in the identified commit may not contain the true root cause, which can mislead developers and complicate the fault localization process.

Recent studies suggest that integrating historical commit information with complementary fault localization techniques can improve localization effectiveness [17], [18], [19]. Notably, Yang et al. [14] revisited SBFL-based techniques and showed that a simple yet widely adopted debugging practice—treating files modified in bug-inducing commits as potential fault candidates—can surprisingly outperform SBFL-based techniques on key localization metrics. Moreover, they observed that BIC-based and SBFL-based approaches are highly complementary, as they tend to localize different subsets of compiler faults. These findings highlight the potential benefits of combining dynamic execution evidence with historical commit information. However, despite this observed complementarity, how to effectively integrate these two heterogeneous sources of information remains an important and largely unexplored challenge.

Our Approach. To address this challenge, we propose DUALTRACK, a hybrid compiler fault localization framework that effectively integrates dynamic execution information from SBFL with historical commit analysis. DUALTRACK operates in a two-layer manner:

- **SBFL Layer:** Computes suspiciousness scores for code elements using SBFL formulas based on execution profiles from passing and failing test cases.
- **Commit Analysis Layer:** Refines the SBFL-based rankings by prioritizing files modified in bug-inducing commits identified through commit analysis.

Specifically, DUALTRACK first applies traditional SBFL techniques to produce an initial ranking of suspicious code elements. It then incorporates commit-level information by emphasizing files modified in bug-inducing commits—commits after which a previously passing test program begins to fail. By integrating these two complementary sources of information, DUALTRACK mitigates the limitations of SBFL and commit-based approaches when used in isolation, and significantly improves fault localization accuracy.

Contributions. We make the following key contributions:

- **Novel Framework:** We propose DUALTRACK, a dual-layer compiler fault localization framework that integrates suspiciousness scores derived from SBFL formulas with historical commit information.
- **Improved Effectiveness:** We conduct an empirical evaluation of DUALTRACK on 120 real-world compiler bugs from GCC and LLVM, demonstrating that it correctly localizes 52% of faulty files at the Top-1 position, substantially outperforming state-of-the-art techniques.
- **Analytical Insights:** We provide an in-depth analysis of the limitations of existing SBFL-based and commit-based approaches and show how DUALTRACK effectively addresses these challenges in complex compiler systems.

Outline. The remainder of this paper is organized as follows. Section II introduces the background on compiler fault localization and outlines our motivation. Section III presents our proposed dual-layer framework, DUALTRACK, in detail. Section IV reports the results of our empirical evaluation, and Section V discusses their implications. Section VI reviews related work, and Section VII concludes the paper.

II. BACKGROUND & MOTIVATION

In this section, we briefly introduce the key concepts of Spectrum-Based Fault Localization (SBFL) and Version Control Systems (VCS) in the context of compiler fault localization, and motivate our hybrid approach by outlining the limitations of existing methods and the advantages of integrating SBFL with commit-based analysis.

A. Spectrum-Based Fault Localization

Spectrum-Based Fault Localization (SBFL) is a widely used technique in software testing and debugging, designed to help developers pinpoint the root causes of software faults [10], [13], [20], [21]. The core principle of SBFL is to collect coverage data from program executions and compute a suspiciousness score for each code element (e.g., statement, function, or file). These scores are calculated based on how frequently each element is executed by failing versus passing test cases. The underlying assumption is that code components executed more often by failing test cases are more likely to contain faults, while those executed mostly by passing test cases are less likely to be faulty [20]. SBFL ranks code elements by their suspiciousness, enabling developers to focus on the most suspicious parts of the code. Commonly used formulae, such as Ochiai [10] and Dstar [11], help quantify the likelihood that a specific code element is responsible for the observed failures. However, SBFL's precision heavily depends on the quality and diversity of the test suite [12], [13]. If there are insufficient failing test cases or if the test cases do not adequately capture the fault, SBFL's accuracy can suffer due to incomplete or misleading coverage data. While increasing the number of failing test cases can improve SBFL's performance [12], constructing such comprehensive test suites is particularly challenging in complex systems like compilers. Compiler behavior can vary significantly due to different optimization flags and configurations, making it difficult to ensure diverse and high-quality test cases. Consequently, SBFL's effectiveness in compiler fault localization is often limited by these factors.

B. Automated Compiler Fault Isolation

Localizing faults in compilers presents unique challenges due to the complexity of modern compiler architectures and

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

the extensive optimizations they employ. A compiler typically consists of multiple stages, including parsing, optimization, and code generation, any of which may introduce subtle and difficult-to-isolate bugs. Moreover, the vast space of possible optimization configurations further exacerbates the fault localization problem.

To address these challenges, several spectrum-based fault localization (SBFL) techniques have been specifically developed for compiler fault isolation. DiWi [3] and RecBi [5], proposed by Chen et al., leverage witness test programs—mutated versions of the original bug-triggering program that do not reproduce the failure but provide complementary coverage information. By jointly analyzing failing executions of the bug-triggering program and passing executions of witness programs, these approaches apply SBFL formulas to compute suspiciousness scores for code elements and identify faulty files in compilers. Building on this idea, Tu et al. proposed LLM4CBI [6], which employs large language models to generate more effective witness programs through carefully designed prompts, thereby further enhancing the effectiveness of SBFL-based compiler fault localization. Subsequently, Liu et al. subsequently introduced ETEM [7], which enhances compiler fault isolation by generating witness test programs using feature-based mutation operators.

In contrast to witness-program-based techniques, Yang et al. proposed ODFL [8] from a configuration-centric perspective. ODFL generates a pair of adversarial compilation configurations for the same test program: a failing configuration and a passing configuration, each enabling only a minimal set of fine-grained optimization options. By analyzing coverage differences across these configurations, ODFL substantially improves the accuracy of compiler fault localization. Building upon this approach, Li et al. introduced MULTICONF [9], which constructs multiple pairs of adversarial compilation configurations tailored for isolating GCC compiler faults, achieving state-of-the-art performance in locating GCC faults.

More recently, Yang et al. revisited compiler fault isolation from a commit-analysis perspective and proposed BASIC [14]. BASIC applies a binary search strategy to identify the bug-inducing commit (BIC) and treats the files modified in the identified BIC as fault candidates. Experimental results show that BASIC often outperforms state-of-the-art SBFL-based techniques in terms of Top-1 and Top-5 localization accuracy. Despite these advances, existing studies have not effectively integrated bug-inducing commit (BIC) information with SBFL-based approaches for compiler fault isolation. Notably, BASIC is highly complementary to SBFL-based techniques, as the two approaches tend to localize largely disjoint subsets of bugs that the other fails to isolate [14]. These observations suggest that systematically combining BIC analysis with SBFL techniques holds significant potential for further improving the effectiveness of compiler fault localization.

C. Commit-Based Fault Analysis

In modern software development, Version Control Systems (VCS), such as Git, are essential for managing code evolution. VCS tools record incremental changes as commits, which encapsulate modified code along with a summary of the changes. This allows developers to trace the history of code modifications, providing valuable insights into the evolution of the codebase. Commit-based fault localization leverages this history to identify the specific **bug-inducing commit (BIC)**—the commit that introduced the bug. Tools like *git bisect* perform a binary search through the commit history to isolate the commit where a previously passing test begins to fail [15], [16], [22]. By identifying the BIC, developers can focus on the files modified in that commit, accelerating the debugging process. Several studies have shown that historical information is helpful for debugging and fault localization, and techniques have been proposed to utilize historical information for these tasks [17], [18]. Commit-based fault localization is particularly effective for identifying bugs introduced by recent changes. However, its effectiveness diminishes when dealing with latent bugs—bugs that preexist in earlier versions of the code but only manifest after subsequent changes [19]. In such cases, a more in-depth analysis of interactions between recent and older code is required.

D. Discussion on Challenges in Existing Techniques

Despite advancements in automated fault localization, developers often find these tools inadequate when the faulty code is not ranked within the top few candidates (e.g., Top-5) [23]. This limitation underscores the need for more precise fault localization techniques that consistently rank defective code elements among the top candidates. Current SBFL techniques for compiler, including DiWi, RecBi, LLM4CBI, ETEM, ODFL, and MULTICONF, have not yet achieved the desired precision in real-world scenarios. For example, even the latest technique, ODFL, only succeeds in localizing 58 out of 120 bugs within the Top-5 ranking, which significantly limits its practical utility. This lack of precision, especially in the context of compiler bugs, highlights the need for more effective solutions that can meet the demands of developers. Commit-based approaches, while intuitive, suffer from their own limitations. Developers often focus on recent commits, assuming that the most recent changes are responsible for the bug [17], [18]. However, this assumption breaks down in complex systems like compilers, where subtle interactions between components can cause delayed bug manifestations. Consequently, relying solely on commit-based methods can result in missed localizations, particularly for bugs that arise from interactions across multiple components.

E. Orthogonality of SBFL and Commit-Based Methods

To better illustrate the limitations of existing approaches, we revisit and summarize the experimental results reported in our prior work [14], which evaluated the fault localization performance of both spectrum-based fault localization (SBFL) and commit-based methods on 60 real-world GCC bugs. Under the same experimental setting, the best-performing SBFL-based technique, ODFL, was compared against a simple commit-based method, HISTTRACK, whic is a variant of BASIC [14]. HistTrack localizes faults by identifying files modified in the bug-inducing

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

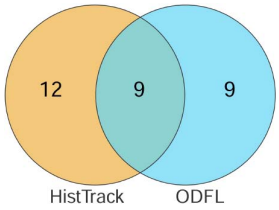


Fig. 1. Relationship between faults correctly localized by HistTrack and ODFL under the top-1 ranked files.

commit and considers a fault successfully localized under the Top-1 metric if the modified file exactly matches the faulty one.

As illustrated in Fig. 1, these results reveal a pronounced orthogonality between spectrum- and commit-based approaches. Specifically, under the Top-1 ranking criterion, ODFL correctly localizes 9 faults that HistTrack fails to identify, whereas HistTrack successfully localizes 12 faults missed by ODFL. This observation indicates that the two approaches capture largely distinct and complementary fault-relevant signals.

These findings highlight the complementary strengths of spectrum- and commit-based techniques: SBFL exploits dynamic execution behavior, while commit-based analysis leverages historical change information. At the same time, the observed orthogonality exposes a fundamental challenge in practice—how to effectively integrate these heterogeneous and potentially conflicting information sources. For instance, files strongly indicated by commit history may receive low suspiciousness scores from SBFL, or vice versa, rendering naive fusion strategies ineffective or even detrimental.

This complementarity, together with the challenge of principled integration, strongly motivates the development of a hybrid fault localization approach. By systematically combining execution-based evidence from SBFL with historical insights from commit analysis, we aim to overcome the limitations of each method in isolation and achieve more accurate and robust compiler fault localization.

III. Approach

In this section, we present DualTrack, a hybrid approach for localizing compiler faults that leverages historical commit information to guide the fault localization process. Fig. 2 illustrates the overall framework of DualTrack.

A. Overview

DualTrack follows a systematic three-step process to efficiently isolate faults within compiler codebases. The overall procedure, outlined in Algorithm 1, combines commit-based analysis with SBFL to identify bug-inducing changes and prioritize relevant source code files. The process consists of three main steps:

1) **Identifying Bug-inducing Commit (Lines 2–3):** This step identifies the bug-inducing commit and determines the set of files modified in that commit. These files, which are more likely to contain the bug, are considered potential candidates. Execution traces further refine the pool of candidate faulty files.
2) **Generating Adversarial Configurations (Lines 4–12):** In this step, adversarial compilation configurations (a set of both failing and passing configurations) are generated. These configurations are derived from the given failing configuration by fine-tuning specific compilation options. The configurations are used for subsequent SBFL analysis.
3) **Ranking Suspicious Files (Lines 13–14):** The final step ranks the suspiciousness of compiler files based on SBFL scores. Files modified in the bug-inducing commit are prioritized in the ranking to increase the likelihood of identifying the faulty files efficiently.

Algorithm 1: Compiler Faults Isolation with DualTrack

Input: BR: Bug report.
Output: $Rank$: Rank of suspicious compiler files.

1 $BadRev, GoodRev, Conf_{fail}, Prog, PassResult \leftarrow$ `InfoRetrieve` (BR)
/* Identify bug-inducing commit */
2 $Commit_{BI}, Commit_{pre} \leftarrow$ `BinarySearch` $(BadRev, GoodRev, Conf_{fail}, Prog)$
3 $ModifiedFiles \leftarrow$ `DiffAnalysis` $(Commit_{BI}, Commit_{pre})$
/* Generate Adversarial Configurations */
4 $Confs_{fail}, Confs_{pass} \leftarrow \varnothing, \varnothing$
5 $FOpts \leftarrow$ `Search` $(Commit_{BI}, Conf_{fail}, Prog)$
6 **foreach** $FOpt \in FOpts$ **do**
7 $Conf \leftarrow$ `GenNewConf` $(Conf_{fail}, FOpt)$
8 **if** `Compile` $(conf) == PassResult$ **then**
9 $Confs_{pass} \leftarrow Confs_{pass} \cup \{Conf\}$
10 **else**
11 $Confs_{fail} \leftarrow Confs_{fail} \cup \{Conf\}$
12 $AdvConfs \leftarrow (Confs_{fail}, Confs_{pass})$
/* Rank Suspicious Files */
13 $Rank \leftarrow$ `SBFL` $(AdvConfs)$
14 $Rank \leftarrow$ `RankAdjust` $(Rank, ModifiedFiles)$
15 **return** $Rank$

16 **Function** BinarySearch$(BadRev, GoodRev, Conf_{fail}, Prog)$
17 **while** $BadRev - GoodRev \neq 1$ **do**
18 $rev \leftarrow$ `Bisect` $(BadRev, GoodRev)$
19 **if** `Compile` $(rev, Conf_{fail}, Prog) ==$ **Fail then**
20 $BadRev \leftarrow rev$
21 **else**
22 $GoodRev \leftarrow rev$
23 **return** $(BadRev, GoodRev)$

B. Identifying Bug-Inducing Commit

A critical aspect of effective fault localization in DualTrack is identifying the commit that introduced the bug, rather than merely the commit where the bug was first reported. Following

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

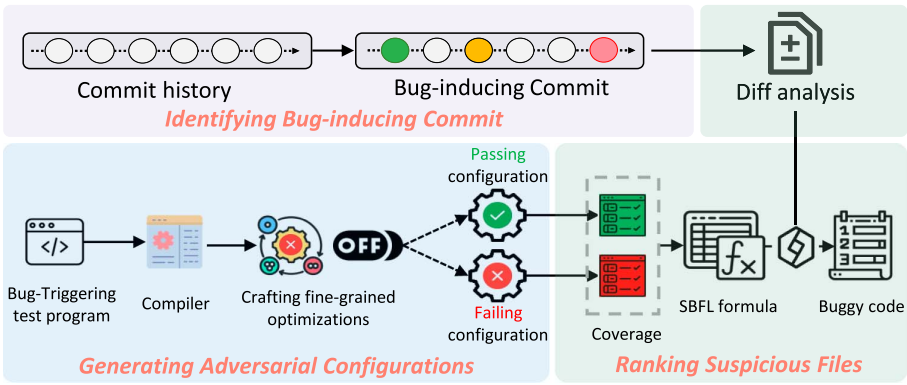


Fig. 2. An overview of DUALTRACK.

established methodologies [17], [18], [19], we define a bug-inducing commit as follows:

Definition 1: Given a bug revealed by a failing test t_f, the bug-inducing commit is the commit such that t_f passes immediately before it, but fails immediately after.

To identify the Bug-Inducing Commit (BIC), we employ a binary search strategy, as described in the function $BinarySearch$ of Algorithm 1. The process begins by extracting relevant information from the bug report, including the compiler version in which the bug was first observed, the revision where the bug was absent, the compiler configuration, and the test program that triggered the bug. If the bug report lacks a precise good revision or date range, we narrow the search by focusing on the release in which the bug first manifested. Using binary search, we systematically test the mainline code between the branch points of the faulty release and the preceding release where the bug was not present. This approach significantly reduces the search space, expediting the identification of the BIC.

Once the BIC is determined, we conduct a differential analysis between this commit and its immediate predecessor to identify the source files modified by the BIC. By comparing the states of these files before and after the commit, DUALTRACK isolates those most likely responsible for introducing the bug—providing a focused set of suspicious files for subsequent fault localization. In our benchmarks, this approach initially flagged an average of 3.95 files per GCC bug and 1.28 files per LLVM bug as suspicious.

However, the identified BIC may not always correspond to the root cause of the bug; sometimes, it merely exposes a pre-existing latent issue. To address this, we broaden our analysis by considering additional compiler files and further refining the candidate pool using execution traces. In our study, 24 out of 60 GCC bugs and 26 out of 60 LLVM bugs fell into this category. Initially, all compiler files are considered as candidates. We then exclude files that were not executed during the bug-triggering compilation. Next, we compare code coverage between failing and passing compilations: files exhibiting differing coverage are retained as suspicious, while those with identical coverage are eliminated. The remaining files comprise the refined pool of candidate faulty files.

C. Generating Adversarial Configurations

Generating adversarial compilation configurations is a cost-effective alternative to mutating test programs [8]. Different compiler configurations can yield varying outcomes for the same test program: some configurations trigger the bug (failing configurations), while others do not (passing configurations). To facilitate the analysis, we define the following terms:

- **Failing Configuration:** A compilation configuration that triggers the bug.
- **Passing Configuration:** A compilation configuration that does not trigger the bug.
- **Adversarial Configurations:** A set of both failing and passing configurations for the same test programs, used to generate spectrum data (i.e., test coverage for both passing and failing cases) for SBFL analysis.

Modern compilers, such as GCC and LLVM, typically use coarse-grained optimization levels (-O0, -O1, -O2, -O3) to balance execution speed and compilation time. Beyond these standard levels, compilers also provide fine-grained options that allow developers to fine-tune specific aspects of code generation. These include loop unrolling, function inlining, common subexpression elimination, dead code elimination, and register allocation. For example, in GCC, function inlining can be explicitly enabled or disabled using the flags *-finline-functions* and *-fno-inline-functions*, respectively. At optimization levels -O2, -O3, and -Os, inlining is enabled by default[1]. Similarly, LLVM provides the *-opt-bisect-limit* option, which allows developers to limit the number of optimization passes, offering precise control over the compilation process[2]. These fine-grained options are invaluable for generating adversarial configurations in fault localization tasks, as they enable the isolation of bugs caused by incorrect compilation passes that lead to erroneous runtime behavior.

In LLVM, optimization passes follow a strict sequential order, enabling efficient identification of bug-triggering passes through binary search. In contrast, GCC presents fine-grained

[1]https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html
[2]https://llvm.org/docs/OptBisect.html

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

compilation options as an unordered set, necessitating a different configuration-processing methodology. Below, we detail the distinct approaches for GCC and LLVM.

GCC. In the context of GCC, DUALTRACK employs a hierarchical approach to generate adversarial configurations by systematically exploring both coarse-grained optimization levels and fine-grained compilation options. The methodology begins by taking the lowest coarse-grained optimization level that triggers the bug as the initial failing configuration. First, following the methodology of ODFL [8], we disable as many non-essential fine-grained options as possible in this configuration to minimize interference. This step, which does not require iterative search, reduces the number of enabled options to an average of only 5 across our benchmark. Subsequently, we systematically test each remaining fine-grained option by individually disabling it to determine if the bug persists. Each test produces either: (a) a new failing configuration if the bug persists, or (2) a passing configuration if the bug is resolved. If this process yields no passing configurations, we take a lower coarse-grained optimization level as the passing configuration. The resulting set of adversarial configurations is then used for SBFL analysis.

LLVM. For LLVM, DUALTRACK employs the *-opt-bisect-limit* option to control the number of optimization passes carefully. This enables the generation of diverse configurations, which are crucial for identifying the root cause of compiler bugs. The *-opt-bisect-limit* option allows us to disable all optimization passes beyond a specified limit without altering the Pass Manager's structure. We retrieve the list of fine-grained optimization passes enabled by default at a coarse-grained optimization level (e.g., -O2) that triggers the bug using the command *clang -O2 -mllvm -opt-bisect-limit=-1*. Binary search is then employed to locate the threshold value that triggers the bug. Initially, the lower bound is set to 0 (disabling all passes), and the upper bound is set to the total number of enabled passes (enabling all passes). The program is compiled using the middle value between the bounds, and the results are examined. If the bug occurs, the upper bound is updated; otherwise, the lower bound is updated. This process continues until the bounds converge, representing the limit value that triggers the bug. If the limit is set below this threshold, the bug will not manifest; otherwise, the bug remains concealed. Therefore, we can easily generate a set of adversarial configurations with the help of the threshold value.

D. Ranking Suspicious Files

The final step in our approach involves ranking the source files based on their likelihood of containing the fault. Files that rank higher are considered more suspicious and are thus prioritized for further inspection.

We utilize Gcov to collect statement-level coverage information for both GCC and LLVM compilers. After compiling a test program with an instrumented compiler, executing it generates the coverage data. The coverage collection for a single configuration is highly efficient, typically completing within seconds. The aggregated coverage spectra from all adversarial configurations form the basis for the subsequent SBFL analysis.

To compute the suspiciousness score for each statement in the code, we utilize SBFL. SBFL operates under the assumption that statements frequently executed by passing test cases are less likely to contain the fault, while statements predominantly executed by failing test cases are more suspicious. This statistical approach has proven effective in various fault localization tasks.

In this work, we adopt the commonly used Ochiai formula [10] to calculate the suspiciousness score for each statement s. The formula, shown in Equation 1, considers three factors: the number of failing test cases (ef_s) that execute statement s, the number of passing test cases (ep_s) that execute statement s, and the number of failing test cases (nf_s) that do not execute statement s. The Ochiai score is given by:

$$score(s) = \frac{ef_s}{\sqrt{(ef_s + nf_s)(ef_s + ep_s)}} \tag{1}$$

After computing the suspiciousness scores for individual statements, DUALTRACK aggregates these scores to determine the overall suspiciousness of each file. Previous studies [3], [5], [6], [8] commonly employ an average aggregation method, where the average score of all statements within a file f is used to compute the file's suspiciousness score, as shown in Equation 2. Here, n_f represents the number of statements in file f covered by the failing test cases.

$$Susp_{avg}(f) = \frac{\sum_{i=1}^{n_f} score(s_i)}{n_f} \tag{2}$$

While the average aggregation method is a reasonable choice, it may fail to fully capture the significance of highly suspicious individual statements. In particular, a file with a few highly suspicious statements could be overlooked if the average score is diluted by many less suspicious statements. Therefore, relying solely on the average may not provide the most accurate assessment of a file's faultiness.

To address this limitation, we propose a weighted aggregation method that combines both the average and the maximum suspiciousness scores of the statements within each file. This approach ensures that files containing highly suspicious statements are appropriately highlighted, even if the average suspiciousness is relatively low. The weighted aggregation is defined in Equation 3, where the weight parameter α controls the balance between the average and maximum scores:

$$Susp(f) = \alpha \cdot \frac{\sum_{i=1}^{n_f} score(s_i)}{n_f} + (1 - \alpha) \cdot \max_i score(s_i) \tag{3}$$

To incorporate historical commit information, we apply a score boost to files modified in the BIC. Specifically, any BIC file that appears within the Top-20 of the initial SBFL ranking has its suspiciousness score increased by a constant value of 1. Formally, for a file f meeting these criteria, its updated score is calculated as:

$$Susp'(f) = Susp(f) + 1 \tag{4}$$

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

This adjustment is motivated by the intuition that files recently modified in the BIC are more likely to be responsible for the observed bug. By incorporating this heuristic, we ensure that files implicated by historical commit data are prioritized, thereby improving the accuracy and efficiency of the fault localization process.

Finally, files are re-ranked based on their adjusted suspiciousness scores, with higher-ranked files indicating a higher likelihood of containing the fault. This ranking helps guide developers' attention to the most suspicious files, facilitating a more targeted and efficient debugging process.

E. Overall Algorithm

Algorithm 1 outlines the overall process of DUALTRACK. The algorithm begins by extracting relevant information from the bug report (Line 1), which includes:

- $BadRev$: The revision where the bug first appeared.
- $GoodRev$: The last known good revision where the bug was absent.
- $Conf_{fail}$: The failing compiler configuration that triggered the bug.
- $Prog$: The test program that produces the failure.
- $PassResult$: The expected correct execution result.

In practice, bug reports often specify a coarse-grained optimization level (e.g., `-O2`) that triggers the bug and, in some cases, another level (e.g., `-O0`) under which the bug does not manifest. We employ an automated script to validate these configurations for each bug. When a report does not provide an explicit failing configuration, the script systematically evaluates standard optimization levels in ascending order to identify the lowest level that reliably reproduces the failure; this level is then used as the initial $Conf_{fail}$. This preprocessing step ensures that a valid and minimal failing configuration is established when bug reports are incomplete.

Consistent with our prior study [14], we adopt the same procedure to identify the bug-inducing commit. Specifically, Line 2 identifies the bug-inducing commit ($Commit_{BI}$) by performing a binary search over the revision range bounded by $BadRev$ and $GoodRev$, and retrieves the commit immediately preceding it ($Commit_{pre}$). Line 3 then conducts a differential analysis between $Commit_{BI}$ and $Commit_{pre}$, yielding a set of modified files ($ModifiedFiles$) that serve as candidate fault locations.

Line 5 searches for fine-grained compilation options ($FOpts$) enabled in the failing configuration. For each such option ($FOpt$), a new configuration ($Conf$) is generated by modifying $Conf_{fail}$ (Line 7). Lines 8–11 compile the program under each generated configuration and classify it as passing ($Confs_{pass}$) or failing ($Confs_{fail}$) based on whether the observed behavior matches the expected result ($PassResult$). All resulting configurations are collectively referred to as adversarial configurations ($AdvConfs$) (Line 12).

Finally, Line 13 computes the suspiciousness score of each file using SBFL based on coverage information collected from the adversarial configurations. The final ranking is then adjusted to prioritize files modified in the bug-inducing commit ($ModifiedFiles$), producing the output of DUALTRACK (Line 14).

IV. EVALUATION

In this study, we address the following research questions:

- **RQ1**: How effective is DUALTRACK?
- **RQ2**: What are the contributions of the key components of DUALTRACK to its overall fault localization performance?
- **RQ3**: How do different statement-to-file score aggregation methods influence the performance of DUALTRACK?
- **RQ4**: How do different SBFL formulae affect the performance of DUALTRACK?
- **RQ5**: What is the remaining line-level investigation effort after DUALTRACK localizes a fault to file-line?

A. Experimental Setup

This section outlines the experimental setup used for evaluating DUALTRACK. We describe the subjects under study, the benchmark datasets used for evaluation, the baseline techniques for comparison, and the metrics employed to assess fault localization performance.

1) Study Subjects: We evaluate DUALTRACK on two widely used compiler infrastructures: GCC and LLVM. Both compilers are crucial to academic research and industrial applications [24], [25], [26], [27], making them ideal candidates for testing compiler fault localization techniques. Their extensive configurability, along with a wide range of use cases, ensures a challenging and diverse environment for evaluating DUALTRACK's accuracy and robustness in fault localization.

2) Benchmark: Our evaluation benchmark comprises 120 real-world compiler optimization bugs, evenly split between GCC and LLVM (60 bugs each). All bugs have been confirmed and fixed by developers, establishing a reliable ground truth for assessing DUALTRACK's fault localization accuracy. Each bug entry is documented with comprehensive details extracted from bug reports, including: the specific optimization level required to trigger the bug, test programs that reproduce the issue, the affected compiler version, and the precise source files responsible for the faulty behavior. To facilitate comparison with prior work, we adopt the GCC dataset introduced in RecBi [5], which has been widely used in subsequent studies such as LLM4CBI [6], ETEM [7], ODFL [8], and MultiConf [9]. For the LLVM dataset, we exclude outdated compiler versions lacking support for the critical `-opt-bisect-limit` option, while incorporating recent bug reports to maintain a total of 60 cases. As summarized in Table I, the benchmark includes 111 wrong-code bugs and 9 ICE-on-valid-code bugs. Table II shows the distribution of the minimum optimization levels required to trigger these bugs: 25 at `-O1`, 29 at `-O2`, 36 at `-O3`, and 30 at `-Os`.

3) Baselines: We compare DUALTRACK against state-of-the-art compiler fault localization methods:

- **DiWi [3]**: DiWi applies local operation mutations on the bug-triggering test program and uses the Metropolis-Hastings algorithm to generate witness test programs.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DISTRIBUTION OF BUG TYPES IN THE BENCHMARK

Bug Type	Count
Wrong Code	111
ICE on Valid Code	9
Total	120

TABLE II
DISTRIBUTION OF THE LOWEST BUG-TRIGGERING OPTIMIZATION LEVELS IN THE BENCHMARK

Optimization Level	Count
O1	25
O2	29
O3	36
Os	30
Total	120

- **RecBi** [5]: RecBi combines structural mutation with reinforcement learning to generate effective witness test programs from a bug-triggering test program.
- **LLM4CBI** [6]: LLM4CBI leverages large language models to produce high-quality witness test programs by generating and selecting effective prompts.
- **ETEM** [7]: ETEM introduces an enhanced program mutation strategy with simplified optimization configurations and more appropriate formula for calculating file suspiciousness.
- **ODFL** [8]: ODFL adjusts fine-grained compilation options to generate both passing and failing compilation configurations, facilitating the collection of comprehensive code coverage information.
- **MultiConf** [9]: MultiConf constructs multiple pairs of adversarial compilation configurations and aggregates suspicious rankings of them using a weighted voting scheme.
- **BASIC** [14]: BASIC applies a binary search between a recent good release and a earlier bad release to pinpoint the bug inducing commit. Files modified in the identified commit are then regarded as faulty candidates.
- **HSFL** [17]: HSFL utilizes version history to assist in fault localization. The key insight is that program entities modified by bug-inducing commits, but rarely by subsequent non-inducing ones, are more likely to be faulty. Although originally developed for general software, we re-implemented HSFL for compilers and included it as a baseline in our evaluation of DUALTRACK.

4) Measurements: To assess the performance of DUALTRACK and the baseline methods, we use several well-established evaluation metrics commonly employed in prior work [3], [5], [6], [7], [8], [9]:

- **Top-N**: This metric measures how many bugs are successfully localized within the top-N positions of the ranked file list. We report results for Top-1, Top-5, Top-10, and Top-20 rankings, indicating how frequently the faulty file appears in the top-N positions.
- **Mean First Rank (MFR)**: MFR calculates the average rank of the first faulty file for each bug. A lower MFR indicates better performance.
- **Mean Average Rank (MAR)**: MAR computes the average rank of all faulty files for each bug. Like MFR, lower MAR values reflect better fault localization performance.

B. RQ1: Comparison of Effectiveness

1) Experimental Setup: To address **RQ1**, we compare DUALTRACK against several baseline methods (DiWi, RecBi, LLM4CBI, ETEM, ODFL, MultiConf, BASIC, and HSFL) using the benchmark datasets described earlier. For the GCC dataset, which is identical to that used in previous studies, we ensure a fair comparison by directly adopting published results. Specifically, we use the results reported in Tu et al.'s recent study [6] (under the default one-hour termination setting) for DiWi, RecBi, and LLM4CBI. The Top-n values for these methods are reported as averages over 10 independent runs (resulting in decimal values), as their underlying techniques involve randomness in witness program generation. For ETEM, ODFL, MultiConf, and BASIC, we use the best performance reported in their original papers. For HSFL, which was not evaluated on GCC in prior work, we re-implemented it for the compiler domain. For the LLVM dataset, which differs from earlier benchmarks, we re-execute all baseline methods ourselves (RecBi, ETEM, ODFL, and HSFL). This ensures correctness, reproducibility, and a fair comparison under the same experimental conditions on the updated dataset. We focus on these baselines because ETEM, ODFL, and HSFL have been shown to outperform other techniques, and RecBi represents a classic witness-program-based approach.

2) Results: Table III presents the results of this comparison. DUALTRACK demonstrates strong performance, localizing 33, 48, 55, and 58 bugs within the Top-1, Top-5, Top-10, and Top-20 positions, respectively, for the 60 GCC bugs, corresponding to success rates of 55%, 80%, 92%, and 97%, respectively. The MFR and MAR values for DUALTRACK on GCC are 3.5 and 4.6, indicating that the first faulty file typically appears within the top 4 files, and all faulty files are ranked within the top 5. For the LLVM dataset, DUALTRACK localizes 29, 40, 45, and 47 bugs within the Top-1, Top-5, Top-10, and Top-20 positions, corresponding to success rates of 48%, 67%, 75%, and 78%. The MFR and MAR values for LLVM are 15.5 and 15.6, respectively. When compared to other approaches, DUALTRACK consistently outperforms state-of-the-art techniques. For example, DUALTRACK isolates 355.9%, 129.6%, 72.2%, 29.2%, and 51.2% more bugs in the Top-1 category than RecBi, ETEM, ODFL, BASIC, and HSFL respectively, with similar gains across the Top-5, Top-10, and Top-20 metrics.

3) Statistical Test Analysis of Results: In alignment with previous work [6], we conduct a Mann-Whitney U-test with a significance level of 0.05 on the total number of bugs located by DUALTRACK versus the state-of-the-art approaches [28]. To mitigate the influence of randomness, we calculate

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EFFECTIVENESS COMPARISON FOR DUALTRACK VS. THE BASELINE APPROACHES

Subject	Technique	Top-1		Top-5		Top-10		Top-20		MFR		MAR	
		Value	⇑(%)	Value	⇑(%)	Value	⇑(%)	Value	⇑(%)	Value	⇑(%)	Value	⇑(%)
GCC	DiWi	5.6	489.3	19.9	141.2	31.3	78.9	41.7	39.1	22.6	-84.5	23.1	-80.1
	RecBi	7.7	328.6	23.9	100.8	34.2	63.7	42.5	36.5	20.5	-82.9	21.1	-78.2
	LLM4CBI	9.3	254.8	24.9	92.8	36.4	53.8	44.8	29.5	15.8	-77.8	16.4	-72.0
	ETEM	20.0	65.0	38.0	26.3	44.0	27.3	54.0	7.4	6.6	-47.0	7.0	-34.3
	ODFL	20.0	65.0	31.0	54.8	42.0	33.3	47.0	23.4	23.1	-84.8	23.8	-80.7
	MultiConf	27.0	22.2	40.0	20.0	51.0	9.8	53.0	9.4	7.4	-52.7	8.5	-45.9
	BASIC	21.0	57.1	35.0	37.1	35.0	60.0	35.0	65.7	–	–	–	–
	HSFL	19.0	73.7	22.0	118.2	30.0	86.7	38.0	52.6	22.7	-84.6	24.2	-81.0
	DUALTRACK	**33.0**	—	**48.0**	—	**56.0**	—	**58.0**	—	**3.5**	—	**4.6**	—
LLVM	RecBi	5.9	391.5	20.2	98.0	30.0	50.0	43.5	8.0	24.6	-37.0	25.3	-38.3
	ETEM	7.0	314.3	13.0	207.7	21.0	114.3	28.0	67.9	42.7	-63.7	43.9	-64.5
	ODFL	16.0	81.3	27.0	48.1	36.0	25.0	47.0	0.0	32.9	-52.9	33.0	-52.7
	BASIC	27.0	7.4	34.0	17.7	34.0	32.4	34.0	38.2	–	–	–	–
	HSFL	22.0	31.8	23.0	73.9	25.0	80.0	30.0	56.7	42.5	-63.5	42.6	-63.4
	DUALTRACK	**29.0**	—	**40.0**	—	**45.0**	—	**47.0**	—	**15.5**	—	**15.6**	—
ALL	RecBi	13.6	355.9	44.1	99.5	64.2	57.3	86.0	22.1	22.6	-58.0	23.2	-56.5
	ETEM	27.0	129.6	51.0	72.5	65.0	55.4	82.0	28.0	24.7	-61.5	25.5	-60.4
	ODFL	36.0	72.2	58.0	51.7	78.0	29.5	94.0	11.7	28.0	-66.1	28.4	-64.4
	BASIC	48.0	29.2	69.0	27.5	69.0	46.4	69.0	52.2	–	–	–	–
	HSFL	41.0	51.2	45.0	95.6	55.0	83.6	68.0	54.4	32.6	-70.9	33.4	-69.8
	DUALTRACK	**62.0**	—	**88.0**	—	**101.0**	—	**105.0**	—	**9.5**	—	**10.1**	—

Columns "⇑" report the improvement rate of DUALTRACK over baseline methods. Note that lower MFR and MAR values indicate better performance.

the effect size of the differences between DUALTRACK and the baseline techniques using Vargha and Delaney's $\hat{A}_{12}$ statistics. Given a performance measure M (e.g., Top-1), the $\hat{A}_{12}$ statistic quantifies the probability that approach A (e.g., DUALTRACK) yields higher M values than approach B (e.g., ODFL). If the two approaches are equivalent, then $\hat{A}_{12} = 0.5$. For instance, $\hat{A}_{12} = 0.9$ indicates that approach A produces higher results 90% of the time compared to approach B.

We utilize the R programming language to perform the Mann-Whitney U-test and obtain the p-value. To calculate the effect size, we employ the following formula:

$$\hat{A}_{12} = \frac{(R1/m) - \left(\frac{m+1}{2}\right)}{n} \tag{5}$$

where $R1$ is the rank sum of the first data group being compared and m represents the number of observations in the first group and n represents the number of observations in the second group. The rank sum is a fundamental component of the Mann-Whitney U-test and is provided by most statistical tools. We compute the $R1$ value using the function "$sum(rank(c(X,Y))[seq_along(X)])$" in R, where X and Y are the datasets containing the observations (i.e., metrics over GCC bugs) of the two approaches being compared.

The overall statistical results are presented in Table IV. A p-value less than 0.05 (often less than 0.001) indicates that DUALTRACK significantly outperforms all existing approaches, including DiWi [3], RecBi [5], LLM4CBI [6], ETEM [7], ODFL [8], and MultiConf [9]. Furthermore, we observe that the effect sizes are 1.0 for all techniques, suggesting that DUALTRACK has a strong likelihood of yielding better results than existing approaches.

TABLE IV
STATISTICAL TEST RESULTS FOR DUALTRACK VS. STATE-OF-THE-ART TECHNIQUES IN LOCATING GCC BUGS

DiWi			RecBi			LLM4CBI		
Metric	p-value	$\hat{A}_{12}$	Metric	p-value	$\hat{A}_{12}$	Metric	p-value	$\hat{A}_{12}$
Top-1	< 0.001	1.00	Top-1	< 0.001	1.00	Top-1	< 0.001	1.00
Top-5	< 0.001	1.00	Top-5	< 0.001	1.00	Top-5	< 0.001	1.00
Top-10	< 0.001	1.00	Top-10	< 0.001	1.00	Top-10	< 0.001	1.00
Top-20	< 0.001	1.00	Top-20	< 0.001	1.00	Top-20	< 0.001	1.00
MFR	< 0.001	1.00	MFR	< 0.001	1.00	MFR	< 0.001	1.00
MAR	< 0.001	1.00	MAR	< 0.001	1.00	MAR	< 0.001	1.00
ETEM			ODFL			MultiConf		
Metric	p-value	$\hat{A}_{12}$	Metric	p-value	$\hat{A}_{12}$	Metric	p-value	$\hat{A}_{12}$
Top-1	< 0.001	1.00	Top-1	< 0.001	1.00	Top-1	< 0.001	1.00
Top-5	< 0.001	1.00	Top-5	< 0.001	1.00	Top-5	< 0.001	1.00
Top-10	< 0.001	1.00	Top-10	< 0.001	1.00	Top-10	< 0.001	1.00
Top-20	< 0.001	1.00	Top-20	< 0.001	1.00	Top-20	< 0.001	1.00
MFR	< 0.001	1.00	MFR	< 0.001	1.00	MFR	< 0.001	1.00
MAR	< 0.001	1.00	MAR	< 0.001	1.00	MAR	< 0.001	1.00

Conclusion. DUALTRACK offers superior precision in fault localization compared to state-of-the-art techniques.

C. RQ2: Contributions of Main Components

1) Experimental Setup: To address **RQ2**, we evaluate the contributions of DUALTRACK's three key aspects: historical version information, SBFL metrics, and adversarial configuration

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EFFECTIVENESS COMPARISON FOR DUALTRACK VS. ITS VARIANTS

Subject	Technique	Top-1		Top-5		Top-10		Top-20		MFR		MAR	
		Value	⇑(%)	Value	⇑(%)	Value	⇑(%)	Value	⇑(%)	Value	⇑(%)	Value	⇑(%)
GCC	HistTrack	21	57.14	35	37.14	35	60.00	35	65.71	—	—	—	—
	SpectrumTrack	17	94.12	38	26.32	52	7.69	58	0.00	5.45	-35.41	6.44	-28.26
	SpectrumTrack$_{Orig}$	0	∞	4	1100.00	20	180.00	34	70.59	29.54	-88.08	31.17	-85.18
	DUALTRACK$_{Orig}$	19	73.68	22	118.18	29	93.10	34	70.59	26.32	-86.63	28.05	-83.53
	DUALTRACK	**33**	—	**48**	—	**56**	—	**58**	—	**3.52**	—	**4.62**	—
LLVM	HistTrack	27	7.41	34	17.65	34	32.35	34	38.24	—	—	—	—
	SpectrumTrack	8	262.50	33	21.21	42	7.14	47	0.00	17.02	-8.99	17.12	-9.05
	SpectrumTrack$_{Orig}$	3	866.67	9	344.44	12	275.00	15	213.33	61.23	-74.70	62.00	-74.89
	DUALTRACK$_{Orig}$	20	45.00	21	90.48	22	104.55	23	104.35	57.54	-73.08	58.69	-73.47
	DUALTRACK	**29**	—	**40**	—	**45**	—	**47**	—	**15.49**	—	**15.57**	—
ALL	HistTrack	48	29.17	69	27.54	69	46.38	69	52.17	—	—	—	—
	SpectrumTrack	25	148.00	71	23.94	94	7.45	105	0.00	11.24	-15.40	11.78	-14.30
	SpectrumTrack$_{Orig}$	3	1966.67	13	576.92	32	215.63	49	114.29	90.77	-89.53	93.17	-89.16
	DUALTRACK$_{Orig}$	39	58.97	43	104.65	51	98.04	57	84.21	83.86	-88.66	86.74	-88.36
	DUALTRACK	**62**	—	**88**	—	**101**	—	**105**	—	**9.51**	—	**10.10**	—

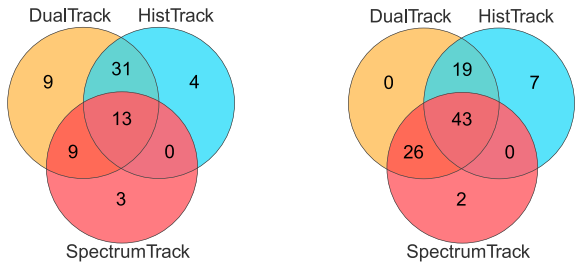
generation. Four variants of DUALTRACK are constructed for this analysis:

- **HistTrack:** A history-only variant that *excludes SBFL metrics*, relying solely on version-control data. It:
  1) Identifies files modified in the bug-inducing commit
  2) If no faulty file exists in this set, fails to localize the fault
  3) Otherwise, ranks files by modification intensity (measured by changed lines)
- **SpectrumTrack:** A spectrum-only variant that *excludes historical data*, operating as follows:
  1) Applies two-step candidate filtering
  2) Generates adversarial configurations
  3) Ranks files using the Ochiai metric
- **SpectrumTrack$_{Orig}$:** A spectrum-only variant that uses *only the original developer-provided configuration* without generating adversarial configurations.
- **DUALTRACK $_{Orig}$:** A hybrid variant that uses both historical and spectral components but employs *only the original configuration* without adversarial configuration generation.

We compare these four variants against the full version of DUALTRACK, which integrates both historical and spectral components with adversarial configuration generation. The results are detailed in Table V. Because HistTrack has missed localizations, its MFR and MAR values are not calculated. Additionally, we analyze the relationship of correct localizations achieved by DUALTRACK and its two individual components, focusing on the Top-1 and Top-5 metrics, as shown in Fig. 3.

2) Results: Table V shows that the full version of DUALTRACK outperforms all variants across all metrics, confirming the benefit of integrating historical information with SBFL metrics enhanced by adversarial configuration generation.

Effectiveness of Configuration Mutation. The comparison between variants with and without configuration mutation reveals its critical importance. SpectrumTrack$_{Orig}$, which uses only the original configuration, achieves only 3 Top-1 localizations across all datasets, while SpectrumTrack with adversarial configuration generation localizes 25 bugs. This represents a remarkable 1966.67 percent improvement. Similarly, DUALTRACK $_{Orig}$ localizes 39 bugs in Top-1, while the full DUALTRACK achieves 62 bugs, corresponding to a 58.97 percent improvement. On average, DUALTRACK generates 6.72 configurations for GCC and 5.78 for LLVM per bug, efficiently exploring the configuration space to obtain diverse execution spectra. These results demonstrate that adversarial configuration generation is essential for effective SBFL and significantly enhances the hybrid approach.



(a) Top-1 (b) Top-5

Fig. 3. Relationship of correct localizations achieved by DUALTRACK and its variants (Top-1 and Top-5).

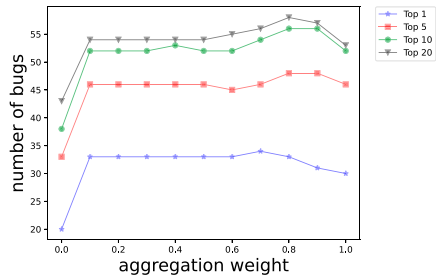
Complementary Nature of Components. Fig. 3 provides further insights into the distribution of correct localizations. In the Top-5 category, we observe substantial overlaps among the methods. Specifically, there are 19 bugs localized only by DUALTRACK and HistTrack (but not by SpectrumTrack), 26 bugs localized only by DUALTRACK and SpectrumTrack (but not by HistTrack), and 43 bugs localized by all three methods. These overlapping cases demonstrate that the fusion strategy in DUALTRACK can effectively harness complementary evidence, enabling it to localize faults that might be missed by only using a single component.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

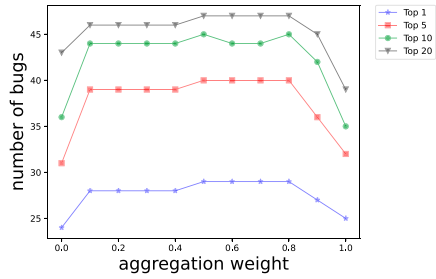
While a small number of faults (7 by HistTrack, 2 by SpectrumTrack) are uniquely localized by one variant in Top-5 but missed by the full DUALTRACK, this does not indicate a conflict between components. Rather, it reflects the conservative nature of our fusion strategy, which favors files highly ranked by both components. As a result, some files highly ranked by only one variant may be slightly demoted, occasionally missing the Top-5.

Crucially, these few exceptions are far outweighed by the many faults localized through the combined approach, including cases like the 9 Top-1 bugs that neither variant finds alone. The considerable overlaps (such as the 19 and 26 cases) further attest to the practical value of the hybrid design. In summary, the integration achieves its intended purpose: leveraging the complementary information from both components to maximize fault localization accuracy.

Conclusion. Each component contributes positively on its own, and their combination markedly improves DUALTRACK's overall performance.



(a) GCC



(b) LLVM

Fig. 4. Impact of different aggregation weight settings.

D. RQ3: Effect of Different Statement-to-File Score Aggregation Strategies

1) Experimental Setup: In **RQ3**, we investigate how different aggregation strategies for deriving file-level suspiciousness scores from statement-level scores affect DUALTRACK's fault localization performance. Prior studies [3], [5], [6], [8], [9] commonly use average aggregation, which may dilute the impact of highly suspicious statements. To overcome this limitation, we propose a weighted aggregation method that combines both the average and the maximum statement-level suspiciousness scores within a file, as detailed in Section III-D. The aggregation is governed by Equation 3, where the parameter $\alpha \in [0.0, 1.0]$ controls the balance between the two scores: $\alpha = 1.0$ corresponds to pure average aggregation, $\alpha = 0.0$ to pure maximum aggregation, and intermediate values produce a weighted blend. By sweeping α from 0.0 to 1.0, we systematically evaluate the effect of the aggregation strategy on localization accuracy.

2) Results: The proposed weighted aggregation significantly improves DUALTRACK's fault localization performance. As shown in Fig. 4, the optimal result is achieved at $\alpha = 0.8$. Performance degrades when using only the average ($\alpha = 1.0$) or only the maximum ($\alpha = 0.0$), confirming that an appropriate balance between the two metrics is essential for accurate file-level fault localization.

3) Generalizability to Baseline Methods: To examine whether the benefit of weighted aggregation is specific to DUALTRACK or generalizable, we also apply it to two baseline SBFL methods: RecBi and ODFL. Table VI presents their Top-5 performance under the same α sweep. Two important observations emerge:

1) **Robust Performance across α Values:** Both RecBi and ODFL exhibit stable Top-5 performance across α values between 0.4 and 0.8, indicating that the method is not sensitive to precise parameter tuning within this range.
2) **Consistent Improvement over Pure Average:** For both methods and datasets, any α between 0.4 and 0.8 consistently outperforms the pure average aggregation ($\alpha = 1.0$). This confirms that incorporating the maximum statement score offers a general and meaningful improvement over traditional averaging.

TABLE VI
TOP-5 PERFORMANCE UNDER DIFFERENT α VALUES

Dataset	Method	$\alpha = 0.4$	$\alpha = 0.5$	$\alpha = 0.6$	$\alpha = 0.8$	$\alpha = 1.0$
GCC	RecBi	26.1	25.2	24.5	25.7	23.9
	ODFL	34	34	34	35	32
	DUALTRACK	46	46	45	48	46
LLVM	RecBi	23.7	24.3	23.6	24.8	20.2
	ODFL	33	33	33	33	30
	DUALTRACK	39	40	40	40	32

Conclusion. The weighted aggregation strategy enhances fault localization performance by effectively combining average and maximum statement suspiciousness. Its stable performance across a wide α range (0.4–0.8) demonstrates robustness, allowing practitioners to adopt the method without fine-grained parameter tuning.

E. RQ4: Effect of Different Formulae

1) Experimental Setup: For **RQ4**, we evaluate how different SBFL formulae affect DUALTRACK's performance. By default, DUALTRACK uses the Ochiai formula [10], which has been

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCE OF DUALTRACK USING DIFFERENT SBFL FORMULAE

Subject	Formula	Top-1	Top-5	Top-10	Top-20	MAR	MFR
GCC	**Ochiai**	**33**	48	**56**	**58**	**3.52**	**4.62**
	Tarantula	32	46	53	55	4.68	5.78
	Dstar	29	46	54	56	4.58	5.66
	Dice	32	47	53	57	4.17	5.28
	Barinel	32	45	50	54	5.30	6.49
	Op2	31	**49**	54	58	4.08	5.23
LLVM	**Ochiai**	**29**	**40**	**45**	**47**	**15.49**	**15.57**
	Tarantula	16	17	20	25	114.09	114.32
	Dstar	16	35	43	47	15.82	15.9
	Dice	**29**	**40**	43	**47**	15.72	15.8
	Barinel	16	17	20	25	114.09	114.32
	Op2	27	35	39	42	23.82	24.06

shown to perform well in prior studies. However, to assess DUALTRACK's robustness, we also examine five other SBFL formulae: Tarantula [29], DStar [11], Dice [30], Barinel [20], and Op2 [31].

2) Results: The results are summarized in Table VII. For GCC bugs, DUALTRACK demonstrates consistent performance across the different formulae. In contrast, for LLVM bugs, the Ochiai formula exhibits superior performance relative to the other formulae, with Dice and Op2 following closely behind. DStar shows comparatively lower efficacy, particularly in terms of Top-1 rankings, while Tarantula and Barinel underperform, especially concerning MAR and MFR. These findings suggest that while DUALTRACK maintains robustness across various formulae, the Ochiai formula remains the most effective option for fault localization in this context. However, it is noteworthy that none of these formulae were specifically designed for compiler bug localization, leaving open the possibility that more effective formulae could be identified in future research.

Conclusion. While DUALTRACK demonstrates adaptability across different SBFL formulae, we recommend the use of the Ochiai formula as the default.

F. RQ5: Remaining Effort for Line-Level Localization

1) Experimental Setup: In **RQ5**, we quantify the remaining debugging effort required after DUALTRACK has localized a fault at the file level. To assess the efficiency of DUALTRACK's ranking, we measure the total lines of code (LOC) that a developer would need to inspect, starting from the top-ranked file, to reach the truly faulty one. This total is defined as:

1) The cumulative LOC of all files ranked above the highest ranking faulty file.
2) The LOC of the faulty file itself.

This metric represents a conservative upper bound for the inspection effort, assuming a developer follows DUALTRACK's ranked list sequentially until the faulty file is encountered. We analyze this effort by reporting its descriptive statistics (maximum, minimum, mean, median) and its distribution across predefined code-size categories.

TABLE VIII
STATISTICS OF REMAINING LINE-LEVEL INVESTIGATION EFFORT AFTER FILE LOCALIZATION BY DUALTRACK

Compiler	Max	Min	Mean	Median	Effort	
					>10KLOC	<5KLOC
GCC	107,199	366	14,112	6,822	38.3% (23/60)	36.7% (22/60)
LLVM	206,536	706	33,051	7,829	48.3% (29/60)	38.3% (23/60)

2) Results: The analysis reveals substantial variation in the remaining investigation effort across bugs and compilers. Aggregate statistics are summarized in Table VIII, with the detailed distribution shown below:

- **GCC:** 6.7% of bugs require inspecting <1K lines; 30.0% require 1K-5K lines; 25.0% require 5K-10K lines; 33.3% require 10K-50K lines; and 5.0% require >50K lines.
- **LLVM:** 13.3% require <1K lines; 25.0% require 1K-5K lines; 13.3% require 5K-10K lines; 23.3% require 10K-50K lines; and 25.0% require >50K lines.

We have the following key observations:

1) **Substantial Remaining Effort:** For a significant portion of bugs (38.3% in GCC, 48.3% in LLVM), developers would need to inspect over 10,000 lines of code to locate the fault, representing considerable manual effort even after successful file-level localization.
2) **Opportunity for Finer-Grained Localization:** These results highlight the potential value of extending fault localization to the statement or line level. Applying SBFL within the localized file could reduce the search space to dozens of suspicious statements, dramatically decreasing manual inspection effort.
3) **Variation Between Compilers:** The remaining effort is notably higher for LLVM bugs (mean: 33,051 LOC) than for GCC bugs (mean: 14,112 LOC), suggesting that more research effort should be devoted to line-level or method-level compiler fault localization.

Conclusion. While DUALTRACK effectively narrows the fault search space to specific files, a substantial line-level investigation effort may remain.

V. DISCUSSION

In this section, we discuss the efficiency, scalability, generality, limitations, and threats to validity of DUALTRACK.

A. Efficiency and Scalability

On average, DUALTRACK requires approximately 1.5 hours to complete fault localization (including BIC identification and generation of adversarial configurations) without pre-installed compiler versions. This runtime is moderately longer than that of methods such as DiWi, RecBi, and LLM4CBI, which typically complete within an hour for witness generation. However,

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

DUALTRACK delivers substantially higher localization accuracy, as shown in Table III. This improvement directly translates to a significant reduction in manual debugging effort. Empirical studies on compiler bug repair indicate that developers often take more than 11 months on average to diagnose and fix a single compiler optimization bug [32]. By sharply narrowing the search space to a small set of suspicious files, DUALTRACK can save developers considerable investigation time, making the additional computational cost a worthwhile investment in practical development workflows.

A core advantage of DUALTRACK is its scalability for large, complex codebases, which is critical for modern compilers like GCC and LLVM. This is achieved through the independent and parallelizable design of its two-layer architecture:

- **Commit-Analysis Layer:** The flexibility of this layer is notable: if the BIC is provided, or if the intermediate commits needed during binary search are pre-built, analysis is nearly instantaneous. Otherwise, binary search over the commit history ensures logarithmic complexity. In practice, maintaining a repository of historical compiler versions or a dedicated debugging server can drastically reduce the setup overhead, making BIC identification highly scalable.
- **SBFL Layer:** Scalability here is enabled by the parallel execution of adversarial configuration tests. Unlike witness-based methods (e.g., DiWi, RecBi, LLM4CBI, ETEM), which require compiling and testing huge number of mutated programs, DUALTRACK varies compiler configurations for a single test program, resulting in a much lighter workload. Each configuration can be executed independently across multiple machines or clusters, allowing substantial speedup through distribution.

The two layers operate concurrently with minimal synchronization, enhancing both performance and deployment flexibility. The commit-analysis layer produces candidate files, while the SBFL layer ranks suspicious files; merging these results is computationally lightweight.

In summary, DUALTRACK strikes a favorable balance between computational overhead and localization effectiveness. Its architecture ensures both efficiency and scalability, making it well-suited for real-world environments.

B. Generality

The hybrid methodology of DUALTRACK, which integrates commit-based historical analysis with dynamic spectrum-based fault localization, extends beyond compilers to a broad range of software systems. This approach is particularly effective for projects exhibiting frequent code changes, large codebases with noisy coverage data, and configurable execution parameters. In practice, DUALTRACK can be adapted to domains such as database systems (by varying cache or buffer settings), distributed systems (by adjusting timeout or replication parameters), and web or mobile applications (by prioritizing recently modified components). The commit-analysis layer transfers directly to any version-controlled project, while adversarial configuration generation adapts to domain-specific tunable parameters. Coverage collection similarly employs appropriate instrumentation tools for the target system. Thus, the principles underlying DUALTRACK generalize to any complex software system that maintains commit history and supports configurable execution, enabling improved fault localization across diverse application domains.

C. Limitations

Despite its strong performance, DUALTRACK has several limitations. First, its effectiveness depends on the availability and quality of detailed commit histories. Second, while adversarial configuration generation is generally efficient, certain bugs may only manifest under highly specific conditions that fall outside the scope of standard optimization levels or fine-grained options typically manipulated by DUALTRACK. Third, the current implementation of DUALTRACK is designed and evaluated under the single fault assumption, which aligns with the typical regression bug scenario in our benchmark. Its performance in scenarios with multiple concurrent faults requires further investigation. Fourth, the present version of DUALTRACK performs fault localization at the file level. In practice, identifying the exact fault after narrowing down to the most suspicious files can still be challenging, particularly when these files contain more than 10,000 lines of code, as is the case for a nontrivial portion of bugs in our dataset. These limitations should be considered when applying DUALTRACK and exploring its applicability in more diverse contexts.

D. Threats to Validity

Internal validity in our study primarily arise from the implementation of DUALTRACK and the baselines. To mitigate these threats, we maintained rigorous experimental controls throughout our evaluation. We compared DUALTRACK against multiple state-of-the-art techniques using either their reported results or our carefully re-implemented versions under consistent experimental conditions. By adopting the same SBFL formula and parameter configurations as previous studies and utilizing the established GCC dataset from RecBi, we minimized potential biases in implementation and configuration, thereby strengthening the causal relationship between our approach and the observed performance improvements.

External validity is mainly influenced by two factors: the choice of compilers and the selection of bugs. While our evaluation demonstrates DUALTRACK's effectiveness on two major compiler infrastructures (GCC and LLVM) with 120 real-world bugs, the generalizability to other compiler systems or different programming languages requires further investigation. The bugs in our benchmark, though diverse in type and optimization levels, may not fully represent all possible compiler fault scenarios. Additionally, while GCC and LLVM are substantial real-world systems, extremely large or specialized compilers might present unique challenges beyond our current experimental scope.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
COMPARISON OF COMPILER FAULT LOCALIZATION TECHNIQUES

Technique	Core Methodology			Key Innovation
	Commit History	Differential Compilations		
		Witness Programs	Adversarial Configurations	
DiWi [3]		✓		First witness program concept
RecBi [5]		✓		RL-guided structural mutations
LLM4CBI [6]		✓		LLM-based witness generation
ETEM [7]		✓		Enhanced mutation with config simplification
ODFL [8]			✓	Fine-grained option manipulation
MULTICONF [9]			✓	Aggregate rankings from multiple configs
BASIC [14]	✓			Bug-inducing commit based analysis
DUALTRACK	✓		✓	**Hybrid dynamic-historical fusion**

For construct validity, we employed well-established fault localization metrics (Top-N, MFR, MAR) that are widely recognized in the literature, ensuring our measurements accurately capture localization effectiveness. The use of developer-confirmed and fixed bugs provides reliable ground truth for evaluation. Our time efficiency measurements, though informative, should be interpreted considering that actual developer experience may vary based on hardware capabilities and specific debugging contexts.

To address conclusion validity, we conducted appropriate statistical tests including Mann-Whitney U-tests with Vargha and Delaney's $\hat{A}_{12}$ effect size measures, demonstrating that DUALTRACK's performance advantages are statistically significant with substantial effect sizes. The consistency of our results across multiple ranking thresholds and both compiler infrastructures, combined with our adherence to established experimental methodologies from prior work, provides robust support for our conclusions. Nevertheless, the practical performance may be influenced by factors such as bug report quality and commit history availability, which warrant consideration in real-world applications.

VI. RELATED WORK

A. Automated Compiler Fault Localization

Automated software fault localization has attracted significant interest, with research spanning spectrum-based techniques [17], [20], [33], slicing-based methods [34], [35], mutation-based approaches [36], [37], [38], and model-based strategies [39], [40]. In the context of compiler fault localization, the literature primarily follows two technical trajectories: witness program-based and configuration-based approaches.

Witness program-based approaches, including **DiWi** [3], **RecBi** [5], **LLM4CBI** [6], and **ETEM** [7], generate both passing and failing test programs to leverage SBFL. However, they often suffer from instability in test generation, high computational costs, and limited diversity in the generated witness programs. Configuration-based approaches, such as **ODFL** [8] and **MultiConf** [9], instead vary compilation options to create adversarial configurations for SBFL analysis, circumventing the need for program mutation. Although these methods reduce reliance on test generation, their accuracy remains limited by the expressiveness of the compiler's configuration space and the difficulty of capturing faults that manifest under complex option combinations. A third, less-explored dimension leverages historical commit data. Techniques like **BASIC** [14] demonstrate that version history can aid fault localization in compilers, yet they often lack integration with dynamic execution analysis and do not fully address the unique behavioral complexity of modern, highly-configurable compilers.

As summarized in Table IX, existing techniques each specialize in a single dimension, including witness program generation (DiWi, RecBi, LLM4CBI, ETEM), adversarial configuration manipulation (ODFL), or commit history analysis (BASIC), and have not effectively integrated these complementary sources of information. This leaves a clear research gap: no prior work has fully fused dynamic execution data and historical commit analysis for compiler fault localization.

Our work, DUALTRACK, is the first to bridge this gap by introducing a hybrid dual-layer framework. In particular:

- **Multi-Dimensional Fusion:** DUALTRACK uniquely integrates commit history with dynamic execution analysis, leveraging both historical context (to prioritize recently changed files) and dynamic behavior (to refine suspiciousness rankings via SBFL).
- **Synergistic Architecture:** By structuring the localization process in two interdependent layers (commit analysis followed by SBFL refinement), DUALTRACK creates a feedback loop that amplifies localization precision, particularly in complex compiler environments.
- **Demonstrated Superiority:** Our evaluation shows that DUALTRACK outperforms all prior techniques, achieving 52% Top-1 localization accuracy. This result is substantially higher than those achieved by ODFL, RecBi, LLM4CBI, ETEM, and BASIC, clearly demonstrating the practical value of fusing historical and dynamic data.

In summary, DUALTRACK advances the state of the art by addressing the shortcomings of prior approaches and showing that integrated, multi-dimensional analysis is essential for effective compiler fault localization.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

B. Bug-Inducing Commit Identification

Identifying bug-inducing commits is a crucial step in debugging and software maintenance [41]. Existing techniques can be grouped into three broad categories [42]:

- **Static Methods:** These methods analyze commit histories or bug reports without executing the code. A notable example is the SZZ algorithm and its derivatives [43], which trace back from a bug-fixing commit to identify prior commits that introduced the bug. It is worth noting that the core heuristic of SZZ is conceptually inspired by the Delta Debugging algorithm, which is a dynamic technique. However, SZZ operates statically by analyzing version control metadata and does not require program execution. These methods rely on the existence of a bug-fixing commit, limiting their applicability in scenarios where no such commit is available.
- **Information Retrieval (IR) Techniques:** These methods treat the identification of bug-inducing commits as a document-retrieval problem, where bug reports are matched with commit logs [40]. Although IR methods avoid the overhead of dynamic techniques (e.g., compilation and testing), their effectiveness is contingent upon the quality and detail of the available bug reports.
- **Dynamic Analysis:** This approach leverages dynamic execution with bug-revealing test inputs [42], [44]. By analyzing code coverage during execution, dynamic analysis helps identify the commit that introduced the bug. While dynamic methods offer improved efficiency, they do not guarantee absolute accuracy, as they may overlook complex interactions between commits and runtime behaviors.

In DUALTRACK, we employ a robust binary search (bisection) strategy over the commit history, utilizing bug-triggering test cases to efficiently identify the bug-inducing commit for each defect. This dynamic approach ensures that the localization process is grounded in actual program behavior, avoiding the pitfalls of static or IR-only methods.

VII. CONCLUSION

In this paper, we presented DUALTRACK, a novel hybrid framework for compiler fault localization that combines spectrum-based and commit-based analysis. Our approach was motivated by the distinct but complementary strengths and limitations of these two methodologies: commit-based methods can overlook faults in unchanged files, while spectrum-based techniques often struggle with ambiguity, especially when coverage is sparse or faults involve complex component interactions. DUALTRACK addresses these challenges through two key innovations: a coverage-guided filtering mechanism that expands the candidate set beyond recently modified files, and a history-aware suspiciousness scoring system that prioritizes files modified in bug-inducing commits during SBFL analysis. This integration not only mitigates the inherent blind spots of each individual approach but also enhances their collective effectiveness. This work opens several promising research directions. Future efforts could investigate designing more effective SBFL formulae specifically for compilers, and refining localization granularity to the function level and line level. Furthermore, applying DUALTRACK's generalizable design to other complex software systems remains a compelling avenue for exploration. DUALTRACK with the result links are publicly available at: https://doi.org/10.5281/zenodo.18093760.

ACKNOWLEDGMENT

We are grateful to the anonymous reviewers and the editorial team for their constructive feedback. We also extend our appreciation to the authors of DiWi, RecBi, LLM4CBI, Odfl, Basic, and MultiConf for generously sharing their repositories.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles Techniques Tools.* 2nd ed. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2006.
[2] D. B. Whalley, "Automatic isolation of compiler errors," *ACM Trans. Program. Lang. Syst. (TOPLAS)*, vol. 16, no. 5, pp. 1648–1659, 1994.
[3] J. Chen, J. Han, P. Sun, L. Zhang, D. Hao, and L. Zhang, "Compiler bug isolation via effective witness test program generation," in *Proc. ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/SIGSOFT FSE)*, Tallinn, Estonia, 2019, pp. 223–234.
[4] D. Zou, J. Liang, Y. Xiong, M. D. Ernst, and L. Zhang, "An empirical study of fault localization families and their combinations," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 332–347, Feb. 2021, doi: 10.1109/TSE.2019.2892102.
[5] J. Chen, H. Ma, and L. Zhang, "Enhanced compiler bug isolation via memoized search," in *Proc. 35th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Melbourne, Australia, 2020, pp. 78–89.
[6] H. Tu, Z. Zhou, H. Jiang, I. N. B. Yusuf, Y. Li, and L. Jiang, "Isolating compiler bugs by generating effective witness programs with large language models," *IEEE Trans. Softw. Eng.*, vol. 50, no. 7, pp. 1768–1788, Jul. 2024, doi: 10.1109/TSE.2024.3397822.
[7] Y. Liu, M. Zhu, J. Dong, J. Yu, and D. Hao, "Compiler bug isolation via enhanced test program mutation," in *Proc. 39th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, New York, NY, USA: ACM, 2024, p., pp. 819–830, doi: 10.1145/3691620.3695074.
[8] Y. Yang, Q. Li, M. Sun, J. Yang, J. Wu, and Y. Zhou, "Isolating compiler faults through differentiated compilation configurations," *IEEE Trans. Softw. Eng.*, vol. 51, no. 6, pp. 1838–1853, Jun. 2025.
[9] Q. Li, Y. Yang, M. Sun, J. Wu, Q. Shi, and Y. Zhou, "Isolating compiler faults via multiple pairs of adversarial compilation configurations," *ACM Trans. Softw. Eng. Methodol.*, to be published.
[10] R. Abreu, P. Zoeteweij, and A. J. Van Gemund, "On the accuracy of spectrum-based fault localization," in *Proc. Testing: Academic Ind. Conf. Pract. Res. Techn.-MUTATION (TAICPART-MUTATION)*, 2007, pp. 89–98.
[11] W. E. Wong, V. Debroy, R. Gao, and Y. Li, "The DStar method for effective software fault localization," *IEEE Trans. Rel.*, vol. 63, no. 1, pp. 290–308, Mar. 2014.
[12] F. Keller, L. Grunske, S. Heiden, A. Filieri, A. van Hoorn, and D. Lo, "A critical evaluation of spectrum-based fault localization techniques on a large-scale software system," in *Proc. IEEE Int. Conf. Softw. Qual., Rel. Secur. (QRS)*, Prague, Czech Republic, 2017, pp. 114–125.
[13] X. Xie, T. Y. Chen, F.-C. Kuo, and B. Xu, "A theoretical analysis of the risk evaluation formulas for spectrum-based fault localization," *ACM Trans. Softw. Eng. Methodol.*, vol. 22, no. 4, pp. 31:1–31:40, Oct. 2013, doi: 10.1145/2522920.2522924.
[14] Y. Yang, Q. Li, M. Sun, J. Wu, and Y. Zhou, "Using a sledgehammer to crack a nut? Revisiting automated compiler fault isolation," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE),* New York, NY, USA: ACM, 2026, doi: 10.1145/3744916.3773254.
[15] L. Online Documentation, "Bisecting LLVM code." Accessed: Apr. 7, 2024. [Online]. Available: https://llvm.org/docs/GitBisecting.html
[16] G. Online Documentation. "How to locate GCC regressions." Accessed: Apr. 7, 2024. [Online]. Available: https://www.gnu.org/software/gcc/bugs/reghunt.html
[17] M. Wen et al., "Historical spectrum based fault localization," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2348–2368, Nov. 2021.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.

[18] M. Wen et al., "Exploring and exploiting the correlations between bug-inducing and bug-fixing commits," in *Proc. ESEC/FSE*, New York, NY, USA: ACM, 2019, pp. 326–337, doi: 10.1145/3338906.3338962.

[19] G. An and S. Yoo, "Reducing the search space of bug inducing commits using failure coverage," in *Proc. ESEC/FSE*, New York, NY, USA: ACM, 2021, pp. 1459–1462, doi: 10.1145/3468264.3473129.

[20] R. Abreu, P. Zoeteweij, and A. J. C. van Gemund, "Spectrum-based multiple fault localization," in *Proc. 24th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Auckland, New Zealand, 2009, pp. 88–99.

[21] R. Abreu, P. Zoeteweij, R. Golsteijn, and A. J. van Gemund, "A practical evaluation of spectrum-based fault localization," *J. Syst. Softw.*, vol. 82, no. 11, pp. 1780–1792, 2009. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121209001319

[22] Accessed: Jul. 20, 2025. [Online]. Available: https://git-scm.com/docs/git-bisect

[23] P. S. Kochhar, X. Xia, D. Lo, and S. Li, "Practitioners' expectations on automated fault localization," in *Proc. 25th Int. Symp. Softw. Testing Anal. (ISSTA)*, New York, NY, USA: ACM, 2016, pp. 165–176, doi: 10.1145/2931037.2931051.

[24] T. Glek and J. Hubicka, "Optimizing real world applications with GCC link time optimization," 2010, *arXiv:1010.2196*.

[25] G. Fursin et al., "Milepost GCC: Machine learning enabled self-tuning compiler," *Int. J. Parallel Program*, vol. 39, no. 3, pp. 296–327, Jun. 2011.

[26] L. P. Cáceres, F. Pagnozzi, A. Franzin, and T. Stützle, "Automatic configuration of GCC using irace," in *Proc. Artif. Evol. - 13th Int. Conf., Évol. Artificielle (EA)*, Paris, France, 2017, pp. 202–216.

[27] D. Berlin, D. Edelsohn, and S. Pop, "High-level loop optimizations for GCC," in *Proc. GCC Developers Summit*, 2004, pp. 37–54.

[28] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proc. 33rd Int. Conf. Softw. Eng. (ICSE)*, 2011, pp. 1–10.

[29] J. A. Jones and M. J. Harrold, "Empirical evaluation of the tarantula automatic fault-localization technique," in *Proc. 20th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Long Beach, CA, USA, 2005, pp. 273–282.

[30] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Trans. Softw. Eng.*, vol. 42, no. 8, pp. 707–740, Aug. 2016.

[31] L. Naish, H. J. Lee, and K. Ramamohanarao, "A model for spectra-based software diagnosis," *ACM Trans. Softw. Eng. Methodol.*, vol. 20, no. 3, pp. 11:1–11:32, 2011.

[32] Z. Zhou, Z. Ren, G. Gao, and H. Jiang, "An empirical study of optimization bugs in GCC and LLVM," *J. Syst. Softw.*, vol. 174, Apr. 2021, Art. no. 110884.

[33] C. M. Tang, W. K. Chan, Y. Yu, and Z. Zhang, "Accuracy graphs of spectrum-based fault localization formulas," *IEEE Trans. Rel.*, vol. 66, no. 2, pp. 403–424, Jun. 2017.

[34] H. Agrawal, R. A. DeMillo, and E. H. Spafford, "Debugging with dynamic slicing and backtracking," *Softw. Pract. Exp.*, vol. 23, no. 6, pp. 589–616, 1993.

[35] X. Zhang, H. He, N. Gupta, and R. Gupta, "Experimental evaluation of using dynamic slices for fault location," in *Proc. 6th Int. Workshop Autom. Debugging (AADEBUG)*, Monterey, CA, USA, 2005, pp. 33–42.

[36] S. Moon, Y. Kim, M. Kim, and S. Yoo, "Ask the mutants: Mutating faulty programs for fault localization," in *Proc. 7th IEEE Int. Conf. Softw. Testing, Verification Validation (ICST)*, Cleveland, Oh, USA, 2014, pp. 153–162.

[37] S. Hong et al., "Mutation-based fault localization for real-world multilingual programs (T)," in *Proc. 30th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Lincoln, NE, USA, 2015, pp. 464–475.

[38] M. Papadakis and Y. L. Traon, "Metallaxis-FL: Mutation-based fault localization," *Softw. Test. Verification Rel.*, vol. 25, nos. 5–7, pp. 605–628, 2015.

[39] R. Abreu, P. Zoeteweij, and A. J. C. van Gemund, "An observation-based model for fault localization," in *Proc. Int. Workshop Dyn. Anal., Held Conjunction ACM SIGSOFT Int. Symp. Softw. Testing Anal. (WODA ISSTA)*, Seattle, WA, USA, 2008, pp. 64–70.

[40] M. Wen, R. Wu, and S. Cheung, "Locus: Locating bugs from software changes," in *Proc. 31st IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, Singapore, 2016, pp. 262–273.

[41] M. Wen et al., "Exploring and exploiting the correlations between bug-inducing and bug-fixing commits," in *Proc. ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng., (ESEC/SIGSOFT FSE)*, Tallinn, Estonia, M. Dumas, D. Pfahl, S. Apel, and A. Russo, Eds. New York, NY, USA: ACM, 2019, pp. 326–337, doi: 10.1145/3338906.3338962.

[42] G. An, J. Hong, N. Kim, and S. Yoo, "Fonte: Finding bug inducing commits from failures," in *Proc.45th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, Melbourne, Australia. Piscataway, NJ, USA*:* IEEE Press, 2023, pp. 589–601, doi: 10.1109/ICSE48619.2023.00059.

[43] J. Sliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" *ACM SIGSOFT Softw. Eng. Notes*, vol. 30, no. 4, pp. 1–5, 2005, doi: 10.1145/1082983.1083147.

[44] G. An and S. Yoo, "Reducing the search space of bug inducing commits using failure coverage," in *Proc. 29th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE),* Athens, Greece, D. Spinellis, G. Gousios, M. Chechik, M. D. Penta, Eds. New York, NY, USA: ACM, 2021, pp. 1459–1462, doi: 10.1145/3468264.3473129.

Authorized licensed use limited to: Nanjing University. Downloaded on October 23,2026 at 05:12:02 UTC from IEEE Xplore. Restrictions apply.